

A concrete extension to the Forge research program

Forge Automatic Arithmetic

Certificate-Producing Quantifier Elimination
and Canonical Witness Synthesis
over Unbounded Naturals

Prepared for Vladimir Reshetnikov
September 2026

Abstract

Forge already proposes an unusually broad collection of proof-producing workers. This article adds a different capability: a compositional decision procedure for first-order formulas over natural numbers with linear arithmetic and selected binary-automatic predicates. Quantifiers range over all natural numbers, not a guessed bit width. The same compiler constructs regular graphs of least witnesses, including the next power of two and the next integer with a prescribed digit-parity relation. Its central engineering object is an exact certificate for existential projection: a locally checkable zero-tail closure combined with a subset-construction receipt. This prevents a subtle but fundamental error in which witnesses longer than their visible inputs disappear.

A standard-library-only Python producer, separately implemented checker, reproducible experiment suite, 90 stored construction certificates, and 643 concrete witness traces accompany the article. The mathematical foundations are classical; the contribution is a focused, executable extension to Forge with explicit semantics and a concrete Lean port plan. Neither the Python checker nor the compiler-to-semantics theorems have been verified in Lean. No new Lean tactic is delivered, and no performance comparison with existing Lean tactics is claimed.

Repository baseline: Forge commit [58ea206bd7ad501b930add568151217bcc7f82a2](#).

Read §1 for the nonduplication boundary, §7 for executed evidence, and §6 for the remaining proof obligations.

Contents

1	The capability delta, not another architecture	3
1.1	What is already present	3
1.2	The intended improvement	3
1.3	What is classical, and what is proposed	4
1.4	Evidence vocabulary	4
2	An exact arithmetic fragment and its digit semantics	5
2.1	Syntax and scoping	5
2.2	Words are representations, not bounded vectors	5
2.3	Linear equalities by exact carries	6
2.4	Inequalities require floor division	6
2.5	Congruences and digit predicates	7
2.6	The source of finiteness	7
3	Quantifier elimination without losing long witnesses	7
3.1	Boolean operations are straightforward only after totalization	7
3.2	The projection error	7
3.3	The correct language operation	8
3.4	A rank and its closed complement certify the exact closure	9
3.5	The complete compiler	9
4	A replayable compilation graph	10
4.1	What the checker is asked to believe	10
4.2	The six node kinds	11
4.3	Why commuting diagrams are enough	11
4.4	Validity and refutation receipts	11
4.5	Resource and decoding discipline	12
4.6	Certificate completeness versus proof compression	12
5	Canonical witness synthesis from regular graphs	13
5.1	Compile the specification, not an ansatz	13
5.2	An evaluator with a concrete termination bound	13
5.3	A regular graph is not automatically a streaming program	14
5.4	Five executed selector families	15
5.5	Towards alternating witness strategies	15
6	Libraries and a concrete Lean integration plan	16
6.1	Use established mathematics without inheriting unsupported trust	16
6.2	The minimal theorem stack	17
6.3	Reification must preserve the actual goal	17

6.4	Proof evaluation and axiom policy	18
6.5	Routing within Forge	18
7	Executed experiments and what they establish	19
7.1	Environment and reproduction	19
7.2	Twenty-five named queries	19
7.3	Independent numerical and quantified oracles	20
7.4	Witnesses and minimality observations	20
7.5	An intentionally broken projection	21
7.6	Mutation and resource controls	21
7.7	The minimization ablation is modest, not a speedup claim	22
7.8	Failed runs and remaining blind spots	22
8	Complexity, next algorithms, and acceptance milestones	22
8.1	Where the cost actually goes	22
8.2	Block existential elimination	23
8.3	Track support and justified miniscoping	23
8.4	Minimize by expected downstream savings	23
8.5	Bitmaps, tables, and proof sharing	24
8.6	An implementation sequence with falsifiable gates	24
9	Conclusion	25
A	Running and extending the bundle	25
B	Certificate schema at a glance	26
C	Review ledger and ownership of claims	27

1 The capability delta, not another architecture

1.1 What is already present

The reviewed Forge snapshot consolidates twenty-seven proposals. Its status ledger already includes polynomial certificates, induction and recurrence synthesis, affine and integer witness extraction, finite-state contracts, closure over words and trees, noncommutative algebra, analytic certificates, games, probabilistic quantities, and pushdown summaries. Its Boolean chapter includes a proof-logging solver for quantifier-free integer difference logic. Repeating any of these as a new idea would not advance the repository. The same ledger distinguishes compiled core-only Lean theorems from unimplemented reflected checkers and explicitly says that there is no installed `forge` tactic [1, 2, 3].

The extension here is *quantification over number encodings*. A state in its automaton summarizes a partially read tuple of binary numerals. Eliminating a track implements an unbounded number quantifier. Boolean combinations and nested quantifiers then operate on these representations compositionally. This is not the existing problem of proving that a matrix observation vanishes on every execution word, nor the existing extraction of a witness from a single linear projection [4].

Axis	Reviewed Forge baseline	Addition developed here
Arithmetic search	Exact certificates and several specialized projection lanes	A first-order compiler with alternating number quantifiers
Infinite domain	Unbounded executions summarized by finite objects	Unbounded numeral lengths, handled with representation-correct projection
Witness language	Affine, lattice, residue and polynomial constructions	Regular least-witness graphs, including binary-automatic relations
Replay	General certificate discipline already designed	Executed local receipts for the complete arithmetic compilation graph
Lean status	Some core lemmas compiled; no tactic	No new kernel result; a specific dependency and porting plan

This comparison is a scoped review of the merged README, status ledger, and relevant closure/Boolean chapters, supported by repository retrieval. It is not a claim to have re-executed or exhaustively audited every frozen proposal. The article develops one substantial additional worker rather than restating the obligation scheduler, general certificate architecture, or source-ownership rules already discussed by Forge.

1.2 The intended improvement

Consider the following families. All variables below range over $\mathbb{N}$.

$$\forall x \exists y \forall z \quad (z \leq x \iff z < y), \quad (1)$$

$$\forall x \exists q, r \quad x = 17q + r \wedge r < 17, \quad (2)$$

$$\forall x \exists y \quad x < y \wedge \text{PowTwo}(y), \quad (3)$$

$$\forall x \exists y \quad x < y \wedge (\text{EvenPop}(x) \iff \text{EvenPop}(y)). \quad (4)$$

Here $\text{PowTwo}(y)$ means that the binary expansion has exactly one set bit, and $\text{EvenPop}(y)$ means that its number of set bits is even. The compiler handles these as formulas; it does

not have a separate programmed proof of each theorem. Changing a coefficient, negating a subformula, or moving a quantifier changes the automata construction automatically. Its resulting machine describes the entire solution relation on the free variables.

For a witness specification $R(x, y)$, the same compiler accepts

$$G_R(x, y) := R(x, y) \wedge \neg \exists z (z < y \wedge R(x, z)).$$

The accepted pairs are exactly the least witnesses. After compilation, a fixed evaluator obtains a witness from the input digits and the graph. There is no fixed affine or polynomial witness template, and no numerical bound on the inputs. This is a meaningful increase in the *expressible synthesis contract*, even before a Lean integration is built.

1.3 What is classical, and what is proposed

Automata-based arithmetic is established mathematics, not a new decision-theory discovery. Binary automata for Presburger constraints appear in the work of Wolper and Boigelot. Walnut implements first-order reasoning about nonnegative integers enriched with automatic predicates, and Mousavi describes its theorem-proving framework [5, 6, 7]. The broad idea “use automata for arithmetic” is therefore not a novelty claim.

There is also directly relevant Lean work. Sheng’s thesis develops verified automata infrastructure and corrects projection for leading-zero encodings. The associated `Aeacu2/Automata` source contains a `project_iff` theorem. It is essential prior work, not a project to overlook while proposing a purportedly first verified automata reasoner [8, 9]. This article uses least-significant-digit-first words and zero *suffixes*; the inspected formalization handles a different ordering/padding interface.

The proposed Forge increment is the following combination: an explicit arithmetic fragment; a producer emitting locally replayable operation receipts; exact padding closure before determinization; a regular least-witness compiler; an executable two-pass witness evaluator; and adversarial, differential, and resource-control evidence. None of these ingredients is claimed as a globally original theorem. Their implementation together is the concrete addition offered to this repository.

1.4 Evidence vocabulary

Evidence status: The Python algorithms and checker were run. The proofs in this article are mathematical arguments, not machine-checked Lean proofs.

A returned Python `valid` certificate means that the Python checker accepted the automata derivation and its accepting invariant. It is not a Lean theorem. A returned counterexample refutes the explicit formula under its declared natural-number semantics; a closed false sentence has an empty free-variable tuple, not a numerical counterexample for every alternation of quantifiers. A resource limit returns `unknown`; it never certifies falsity.

The delivered code does not change Forge, Leant or Djex on GitHub. It is an independently packaged research extension intended for review and integration. The absence of a Lean executable in this environment prevented elaboration, reflection, and native-tactic experiments. This is a limitation of the delivered evidence, not a claim that the required formalization is impossible.

2 An exact arithmetic fragment and its digit semantics

2.1 Syntax and scoping

A context is an ordered list of distinct variable names $\Gamma = (x_0, \dots, x_{k-1})$. Variables denote natural numbers, while coefficients and linear expressions are interpreted in the integers. Atomic formulas are

$$\sum_i a_i x_i = b, \quad \sum_i a_i x_i \leq b, \quad \sum_i a_i x_i \equiv b \pmod{m},$$

with $a_i, b \in \mathbb{Z}$ and $m \in \mathbb{N}_{>0}$, together with digit parity, powers of two, and the ternary graphs of bitwise AND and XOR. Formulas are closed under Boolean operations and existential quantification. Universal quantification is the syntax expansion $\forall x \phi := \neg \exists x \neg \phi$.

A binder appends a fresh variable to the context. This gives a precise projection convention: the eliminated variable is always the last track. Shadowing an active name is rejected; independent sibling scopes may reuse a name. The prototype uses explicit named syntax, not a parser for Lean text. A production reifier should use scoped indices or actual Lean local-expression identities, rather than trusting names.

Multiplication by an integer constant is supported. Multiplication of two variables, exponentiation with a variable exponent as a binary relation, arbitrary recursive predicates, and unrestricted integer quantifiers are not implemented. Recognizing whether one number is a power of two does *not* amount to recognizing the two-variable graph $y = 2^x$. Likewise, a signed linear coefficient is not Lean’s truncated natural subtraction. A goal containing `Nat.sub` needs a proved case split or an exact normalization theorem before entering this fragment.

2.2 Words are representations, not bounded vectors

Let $\Sigma_k = \{0, 1\}^k$. A word $w = d_0 \cdots d_{n-1}$ is read from least significant to most significant digit. Its value on track i is

$$\text{val}_i(w) = \sum_{j=0}^{n-1} 2^j d_j[i]. \quad (5)$$

The empty word represents the all-zero tuple. Appending any number of all-zero letters does not change the tuple:

$$\text{val}(w0_k^t) = \text{val}(w).$$

Every natural tuple has at least one encoding. The integer used for a letter in JSON is $\sum_i 2^i d[i]$; this is distinct from the place value 2^j of its position in the word.

Definition 2.1 (Correct representation). A deterministic automaton A_ϕ represents ϕ in context Γ when, for *every* finite word,

$$\text{Acc}(A_\phi, w) \iff \text{Sem}(\phi, \text{val}(w)). \quad (6)$$

Consequently its accepted language is invariant under appending zero letters.

The quantifier “every word” is deliberate. Correctness only on canonical encodings is insufficient for unrestricted complementation and projection. The implementation avoids a separate canonical-language side condition by representing a tuple on all of its zero-padded encodings.

2.3 Linear equalities by exact carries

For $\sum_i a_i x_i = b$, start with carry $c_0 = -b$. Given a letter d , put $s(d) = \sum_i a_i d[i]$. The transition is

$$c \mapsto \begin{cases} (c + s(d))/2, & c + s(d) \text{ is even,} \\ \perp, & c + s(d) \text{ is odd,} \end{cases} \quad \perp \mapsto \perp. \quad (7)$$

The accepting condition is $c = 0$. No floating-point operations occur.

Lemma 2.2 (Equality carry invariant). *Before the dead state is entered, after j digits,*

$$2^j c_j = -b + \sum_i a_i \text{val}_i(d_0 \cdots d_{j-1}).$$

The automaton satisfies (6) for the equality atom.

Proof. The initial equation is immediate. Multiplying the transition equation by 2^{j+1} adds exactly the contribution $2^j s(d_j)$. An odd residual at a step cannot be repaired by later digits: all their contributions are divisible by the next power of two. If no such obstruction occurs, after the complete word the displayed invariant says that the target equality holds exactly when the final carry is zero. $\square$

A finite bound follows without imposing a numeral width. Set $M = \max(1, |b|, \sum_i |a_i|)$. The initial carry lies in $[-M, M]$; every nondead successor does too. Thus the possible carries plus $\perp$ form a finite universe. The producer explores only reachable carries, rather than allocating this entire interval.

2.4 Inequalities require floor division

For $\sum_i a_i x_i \leq b$, start at $c_0 = -b - 1$, update

$$c' = \left\lfloor \frac{c + s(d)}{2} \right\rfloor, \quad \text{accept precisely when } c < 0. \quad (8)$$

The extra -1 changes the non-strict integer inequality into a strict negativity test. It is not optional.

Lemma 2.3 (Inequality carry invariant). *After j digits the carry is*

$$c_j = \left\lfloor \frac{-b - 1 + \sum_i a_i \text{val}_i(d_0 \cdots d_{j-1})}{2^j} \right\rfloor.$$

Hence the automaton represents the inequality atom.

Proof. For integral s , the identity $\lfloor (\lfloor u/2^j \rfloor + s)/2 \rfloor = \lfloor (u + 2^j s)/2^{j+1} \rfloor$ proves the invariant by induction. At the end, division by a positive integer has a negative floor exactly when its integer numerator is negative. That numerator is $\sum_i a_i x_i - b - 1$, which is negative exactly when $\sum_i a_i x_i \leq b$. $\square$

Python's `//` with positive denominator has the required floor semantics, including negative carries. A port must prove that its integer-division operator has this behavior; truncation toward zero changes the accepted language. The interval bound now uses $M = \max(1, |b + 1|, \sum_i |a_i|)$. Since $-M \leq (c + s)/2 \leq M$, flooring preserves the same integral bounds.

For example, $x \leq 0$ begins at -1 . Reading a zero leaves the carry -1 , accepting the zero tuple under arbitrarily much padding. Reading a one gives zero and rejects. This one-step boundary case detects both a missing -1 and the wrong division convention.

2.5 Congruences and digit predicates

For $\sum_i a_i x_i \equiv b \pmod{m}$, use a pair $(r, t) \in (\mathbb{Z}/m\mathbb{Z})^2$:

$$(r_0, t_0) = (-b \bmod m, 1 \bmod m), \quad (r, t) \xrightarrow{d} (r + ts(d) \bmod m, 2t \bmod m).$$

After j digits, r is the current residual modulo m , and $t = 2^j \bmod m$. Accept when $r = 0$. At most m^2 states exist, often far fewer are reachable, and $m = 1$ is handled without a special mathematical exception. Negative coefficients use exact modular arithmetic.

Digit parity needs two states, toggling for each set bit on the relevant track. Powers of two need three states recording zero, one, or at least two set bits. The bitwise graphs use a persistent failure flag: each letter must satisfy $d_z = d_x \wedge d_y$ or $d_z = d_x \text{ xor } d_y$. These tests are preserved by zero padding. Boolean constants use one state.

The prototype hard-codes this small atom inventory and independently checks every label transition. A future registry of user-defined automata must require an actual theorem identifying the accepted language with the requested Lean predicate. A name such as “prime” attached to an automaton is not such a theorem. Nor should the finite truth table of a few values be admitted as an atomic semantic bridge.

2.6 The source of finiteness

The number domain is infinite, but an atom stores only a carry, residue, or bounded digit observation. A compound formula stores finite combinations of these observations. Input length remains unrestricted throughout. There is no operation that replaces $\forall x : \mathbb{N}$ by $\forall x < 2^W$, and no inference that a theorem holds because a sufficiently large-looking set of integers was tested.

3 Quantifier elimination without losing long witnesses

3.1 Boolean operations are straightforward only after totalization

Conjunction, disjunction and equivalence are synchronous products. A product state records the two component states; its successor records their successors on the same letter. Acceptance applies the corresponding Boolean operator to their acceptance bits. The producer explores only reachable pairs.

Negation preserves the initial state and transitions and flips every acceptance bit. This operation is correct because every DFA is complete: every state has exactly one successor for every letter. Flipping the finals of a partial machine is not a valid implementation of complement unless the missing transitions first acquire a rejecting sink. The checker rejects incomplete transition tables rather than guessing their intended meaning.

An explicit quotient map may reduce a DFA between operations. Minimality is irrelevant to correctness. The relevant equations are preservation of the start state, transitions, and acceptance. The certificate checker tests those equations, not the producer’s partition-refinement history.

3.2 The projection error

Let A represent $\phi(\vec{x}, y)$. On a visible letter $d \in \Sigma_k$, projecting away the last track creates NFA successors

$$\Delta(q, d) = \{\delta(q, (d, 0)), \delta(q, (d, 1))\}.$$

Ordinary letterwise projection guesses exactly one hidden bit per visible input bit. It therefore recognizes witnesses fitting in the *chosen representation length*, not all natural-number witnesses.

Take $\phi(x, y) \equiv y = 2x + 1$. The minimal encoding of $x = 1$ has one bit, while its only witness $y = 3$ needs two. The same-length projection rejects the one-bit input, but accepts a zero-padded two-bit representation of the same input. At $x = 0$, the empty encoding leaves no hidden bit positions at all, despite the witness $y = 1$.

This is a semantic error, not a bad heuristic. It can reverse a theorem's truth value after complementation. The leading-zero issue is explicitly addressed in Sheng's formalization [8, 9]; the least-significant-first implementation here must solve its zero-suffix counterpart.

3.3 The correct language operation

Write π for deletion of the last track. The desired existential language is

$$L_{\exists y \phi} = \{w : \exists t \geq 0, w0_k^t \in \pi(L_A)\} = \pi(L_A)/0_k^*. \quad (9)$$

The quotient by the regular zero-tail language is what allows a hidden value longer than the visible tuple. No arbitrary upper bound on the hidden integer appears in this equation.

Instead of performing the correction after an exponential subset construction, compute it on the original DFA states. Define

$$E = \mu X (F \cup \{q : \exists b \in \{0, 1\}, \delta(q, (0_k, b)) \in X\}), \quad (10)$$

where F is the original accepting set. Thus $q \in E$ means that some finite sequence of zero-visible letters and chosen hidden bits reaches F . Reverse breadth-first search computes this set using only two outgoing edges per original state.

Now determinize the projected NFA. A subset state S has transition

$$S \xrightarrow{d} \bigcup_{q \in S, b \in \{0, 1\}} \{\delta(q, (d, b))\}, \quad (11)$$

starts at $\{q_0\}$, and accepts exactly when $S \cap E \neq \emptyset$. Zero-tail saturation changes the final test, not these ordinary transitions.

Theorem 3.1 (Existential compilation). *If A represents ϕ on every encoding, the subset automaton with transitions (11) and final condition $S \cap E \neq \emptyset$ represents $\exists y \phi$ on every encoding.*

Proof. After reading a visible word w , induction on its length shows that its subset consists exactly of source states obtainable by choosing a hidden bit at every position of w . If that subset intersects E , some such state has a finite zero-visible tail to an original final state. Concatenating the choices gives a joint word whose visible value is $\text{val}(w)$ and whose hidden value is some y . Source correctness yields $\phi(\text{val}(w), y)$.

Conversely, suppose $\phi(\text{val}(w), y)$. Choose a length at least both $|w|$ and the bit length of y . Extend w with zeros to that length and insert the digits of y on the hidden track, padded with zeros if necessary. The joint word represents a satisfying tuple, so the source accepts it. The state after the first $|w|$ positions lies in the projected subset and reaches an original final along the remaining zero-visible tail. It belongs to E , proving acceptance of the original, possibly very short, visible word. $\square$

3.4 A rank and its closed complement certify the exact closure

The producer returns, for each source state, either `null` or a natural rank $r(q)$. Each positive-rank state also carries a bit $b(q)$. Let E_r be the non-null states. The checker requires:

$$F \subseteq E_r, \quad (12)$$

$$r(q) = 0 \implies q \in F, \quad (13)$$

$$r(q) > 0 \implies r(\delta(q, (0_k, b(q)))) < r(q), \quad (14)$$

$$q \notin E_r \implies \delta(q, (0_k, 0)), \delta(q, (0_k, 1)) \notin E_r. \quad (15)$$

Every referenced successor rank must be non-null. The wire format bounds all ranks below the source state count.

Lemma 3.2 (Exactness of the tail receipt). *Conditions (12)–(15) imply $E_r = E$.*

Proof. Rank induction proves $E_r \subseteq E$: rank zero is already final, and a positive rank takes a recorded edge to a smaller rank. Conversely, the complement of E_r cannot reach E_r by any zero-visible edge, and contains no original final. It therefore contains no state that reaches F , so $E \subseteq E_r$. $\square$

Both halves matter. Descent alone prevents spurious witnesses but permits legitimate states to be omitted. That omission becomes unsound when a later negation turns false rejection into acceptance. Complement closure alone can admit a useless cycle disconnected from the finals. The combination establishes an *equality of languages*, which is the invariant required under arbitrary quantifier alternation.

Corollary 3.3 (Bounded tail, unbounded input). *If a source automaton with Q states has any suitable zero-visible tail, it has one of length at most $Q - 1$.*

Proof. A shortest path to an accepting state repeats no state. Alternatively, the producer's reverse breadth-first ranks are shortest-path distances and are below Q . $\square$

This is a bound derived from the already constructed finite source graph. It is not a guessed word-length cutoff. It bounds the *extra* digits a witness may need beyond a particular visible encoding, and the input itself can have arbitrarily many digits.

3.5 The complete compiler

```

compile(context, formula):
  if formula is an atom:
    explore exact finite carry / residue / digit states
    emit an atomic-label receipt
  if formula is NOT child:
    compile child; complement the complete DFA
  if formula is left AND/OR/IFF right:
    compile both; explore the reachable synchronous product
    emit state-pair labels
  if formula is EXISTS x, child:
    compile child in context ++ [x]
    compute zero-visible tail ranks on the child DFA
    determinize projected transitions
    accept a subset exactly when it meets the tail closure
    emit ranks, choices, and subset labels
  optionally minimize; emit a commuting quotient map
  return the resulting DFA node

```

The implementation memoizes by the full ordered context and formula, not merely a printed subexpression. Consecutive universal quantifiers are represented by the same complement/projection mechanisms, with no privileged universal rule. There is also no requirement to transform the formula into prenex normal form, which can unnecessarily enlarge intermediate dependencies.

Theorem 3.4 (Fragment decision procedure). *Ignoring operational cutoffs, compilation terminates for every finite formula in the supported fragment and produces an automaton satisfying (6). Universally closing its free variables is decidable by finite-state reachability.*

Proof. Each atom has a finite state space by the bounds above. Products, complements, finite quotients and powerset constructions preserve finiteness. Recursion follows the finite formula syntax. Semantic correctness follows by induction, with Theorem 3.1 handling existential nodes and ordinary Boolean semantics handling the others. Every natural tuple has an encoding, so all free-variable assignments satisfy the formula exactly when every reachable state is accepting. $\square$

This is automata-valued quantifier elimination: the result is a finite representation of the solution relation, not necessarily a quantifier-free formula in the input syntax. Operationally the producer has limits; therefore its API is a partial procedure with explicit **unknown**, not a promise to decide every supported input under a fixed budget.

4 A replayable compilation graph

4.1 What the checker is asked to believe

The certificate is a directed acyclic array of nodes. A node binds together an ordered variable context, an explicit formula, a complete DFA, and a construction receipt. Child indices must refer strictly backward. A separately supplied *expected query* must match both the bundle query and the root judgment. The checker does not accept a theorem statement merely because the producer placed it in the certificate.

The wire schema is **forge-auto-1**. A DFA has exactly four keys: **k**, **start**, **trans**, and **final**. States are array indices. Row lengths must be 2^k ; every target must be a valid state index; every final flag must be a Boolean. Formula coefficients and indices must be genuine integers: in Python, **True** == 1, so ordinary permissive type checks and structural comparisons are unsuitable at this boundary.

The checker independently implements atomic arithmetic and the local construction equations. It imports neither the producer nor its formula helpers. It does not rerun breadth-first exploration, partition refinement, or the quantifier compiler. Both programs nevertheless share the *mathematical specification*, and both are Python. Independence reduces correlated implementation risks; it does not constitute formal verification.

4.2 The six node kinds

Node	Receipt	Local checks
Atom	Carry, residue, or digit labels	Initial label, each transition equation, exact final condition
Not	Child index	Same context and transitions; formula negated and finals flipped
Product	Two children; state pairs	Correct initial pair, transition pairs, Boolean acceptance
Quotient	Child; total state map	Start, transitions, and finals commute with the map
Exists	Child; subsets; ranks; bit choices	Exact projected transitions and exact zero-tail closure
Root verdict	Accepting invariant or rejecting word	Validity closure or concrete rejecting run and tuple decoding

The root verdict is not itself a compilation node, but is included here to show the whole acceptance boundary. Any unknown node kind, forward dependency, unsupported atom, context mismatch, or query mismatch is rejected.

4.3 Why commuting diagrams are enough

For a quotient map $h : Q_A \rightarrow Q_B$, the checker verifies

$$h(q_A) = q_B, \quad h(\delta_A(q, d)) = \delta_B(h(q), d), \quad [q \in F_A] = [h(q) \in F_B].$$

Induction on words gives equal acceptance from the two initial states. The checker does not need to establish that B is minimal or that h is injective. Surjectivity is also unnecessary: any extra target state outside the reachable image is irrelevant to start-state behavior. This permits different search implementations to emit different valid quotients.

The pair labels for products and subset labels for projection play the same role. They describe the semantic meaning of each stored state and satisfy the exact start/step/final equations. Duplicate labels would not invalidate the mathematical argument, although the producer normally constructs canonical reachable states. The checker requires canonical sorted subsets to keep the wire format deterministic and easy to inspect; uniqueness across all subset-state labels is not a separate proof obligation.

4.4 Validity and refutation receipts

To certify that ϕ holds for all assignments of its free variables, supply a set I of states satisfying

$$q_0 \in I, \quad I \subseteq F, \quad q \in I \implies \delta(q, d) \in I \quad (d \in \Sigma_k).$$

Every run remains in I , and therefore accepts. The producer uses the reachable set, but any larger transition-closed accepting set works.

A refutation supplies a finite word rejected by the root DFA and the exact tuple decoded from that word. The checker follows the supplied letters directly. The producer finds a shortest rejecting *word* by breadth-first search; it does not promise a numerically least tuple, and the checker need not validate shortestness.

For a closed sentence, there are zero visible tracks. The alphabet still contains its unique empty letter, and every word denotes the empty tuple. A rejecting run then certifies that the sentence

is false. For example, the false sentence $\exists m \forall n (n \leq m)$ has no free numerical tuple to display. Its refutation certificate is a semantic automata derivation, not a synthesized adversary strategy for the quantifier game.

Theorem 4.1 (Mathematical soundness of the receipt rules). *Suppose a finite certificate graph satisfies all the specified typing and local receipt conditions. Its root automaton represents its root formula. An accepted validity receipt proves its universal closure under the fragment semantics; an accepted rejecting-word receipt gives a falsifying assignment to the free variables.*

Proof. Induct over the backward-indexed graph. Atomic labels satisfy the invariants of §2. Boolean nodes preserve semantics directly. The commuting equations preserve accepted languages at quotient nodes. At existential nodes, Lemma 3.2 establishes the exact tail set, the subset equations establish the exact projected runs, and Theorem 3.1 supplies the quantified semantics. Root query binding identifies the resulting judgment with the expected one. A second word induction proves the accepting-invariant verdict; direct evaluation proves the rejecting-word verdict. $\square$

Implementation qualification. The theorem is about the stated receipt conditions. Establishing that the implementation of `checker.py` enforces exactly those conditions is a separate software-correctness task. The experiments test it extensively, but there is no mechanized proof of that implementation or of its parser.

4.5 Resource and decoding discipline

The producer defaults to at most 8 active tracks, 8,192 states per constructed machine, 4,096 certificate nodes, and 2,000,000 accumulated transition cells. The checker separately limits contexts to 8 tracks, graphs to 4,096 nodes, tables to 20,000 states each, and their accumulated cells to 2,000,000. The checker also limits formula depth and its CLI rejects a certificate file larger than 100 MB.

These are safeguards, not a hostile-input security audit. They do not fully bound arbitrary-precision integer size, JSON nesting before parsing, every allocation, or total operating-system resource use. Production use still requires byte and integer-bit limits before expensive decoding, isolated processes, cancellation, and memory accounting. A resource refusal must remain distinct from both semantic refutation and a malformed-certificate diagnosis.

The producer’s budget exception is caught by its CLI and serialized as `unknown`. The proof checker rejects bundles carrying such a status. There is no conversion from “the search could not build the machine” to “the formula is false.”

4.6 Certificate completeness versus proof compression

For every successful unrestricted compilation, the producer can emit the local labels, maps and ranks used in the construction. Thus these receipt rules do not introduce a further logical incompleteness within the fragment. They do impose a potentially large storage cost: a small minimized root may have a much larger derivation graph.

Keeping the pre-minimization source is necessary for the current quotient receipt. A future checker could accept independently certified bisimulation or language-inclusion witnesses to avoid retaining some intermediate tables, but that is not implemented here. The existing format deliberately favors transparent local checks over aggressive compression. Report the total certificate size, not just the size of its final automaton.

5 Canonical witness synthesis from regular graphs

5.1 Compile the specification, not an ansatz

Suppose $R(\vec{x}, y)$ is any formula in the supported fragment, including formulas with internal quantifiers. Introduce a genuinely fresh variable z , rename the free output occurrences capture-avoidantly, and form

$$G_R(\vec{x}, y) = R(\vec{x}, y) \wedge \neg \exists z (z < y \wedge R(\vec{x}, z)). \quad (16)$$

The ordering atom is linear: $z - y \leq -1$. The automatic compiler therefore constructs a DFA for G_R .

Theorem 5.1 (Regular least choice). *For every $\vec{x}$, the relation $G_R(\vec{x}, -)$ is either empty or contains exactly one natural number. It is nonempty exactly when $R(\vec{x}, -)$ is nonempty. If $\forall \vec{x} \exists y R(\vec{x}, y)$, its automaton represents the graph of a total function selecting the least witness.*

Proof. Every nonempty subset of $\mathbb{N}$ has a least element. Equation (16) accepts that element and excludes every greater satisfying element. Two accepted values cannot be distinct, because the lesser would contradict the exclusion condition for the greater. Regularity follows by the already established Boolean and existential compiler operations. $\square$

This is a useful synthesis theorem rather than merely an existence argument: the graph has an effective finite representation, and the next subsection gives an executable evaluator. No claim is made that every resulting function has a small graph or a simple closed form.

The public helper rejects a proposed fresh name already free in the relation and rejects renaming that would capture it under an inner binder. A caller must also keep the fresh name distinct from the surrounding input context. The compiler checks active-context shadowing independently. A Lean implementation should make freshness structural through binder indices rather than treating it as a naming convention.

5.2 An evaluator with a concrete termination bound

Let B represent a witness relation with the output on its last track. Fix a visible input tuple, and let w be its minimal common-length binary encoding. The producer's evaluator works as follows.

```

compute zero-visible tail ranks for B
reachable := {B.start}
for each visible letter d of w:
  next := empty set
  for each q in reachable and b in {0,1}:
    t := B.transition(q, (d,b))
    if t not yet in next:
      remember predecessor (q,b) for this position and t
    insert t in next
  reachable := next
choose q in reachable having a finite tail rank
if there is no such q: return no witness
backtrack remembered predecessors to recover prefix output bits
follow decreasing tail ranks to recover extra output bits
return the decoded output, joint word, and complete state trace

```

Only one predecessor is needed per state and input position. Any discarded history reaches the same state and therefore has exactly the same possible continuations. With an arbitrary

relation the evaluator returns some witness; with G_R , every accepting history decodes to the unique least one.

Proposition 5.2 (Evaluator correctness and bound). *For a Q -state source graph and an input encoding of length n , the evaluator returns a witness exactly when one exists. Its returned joint word has length at most $n + Q - 1$. Its graph-operation cost is $O((n + 1)Q)$, with $O(nQ + Q)$ predecessor and rank storage.*

Proof. Induction over the visible positions establishes the exact reachable-state set for all hidden prefixes. The tail set is exact by Lemma 3.2, so its intersection with the final prefix set is nonempty exactly when the visible input has a witness. Backtracking reconstructs a valid prefix run, and decreasing ranks reconstruct an accepting tail of length at most $Q - 1$. Each visible position considers at most $2Q$ transitions; the tail graph also has $2Q$ edges. $\square$

The bound counts graph operations, not the bit complexity of repeatedly constructing large Python integers or parsing their decimal representations. Nor is it the cost of compiling the graph in the first place. Graph compilation may be expensive; evaluation on a fixed verified graph is the comparatively cheap operation.

The concrete witness checker follows the supplied state trace and decodes the tuple independently. It requires an *already verified* source automaton. A valid run on an unbound or wrongly labeled automaton proves nothing about the original witness specification.

Corollary 5.3 (A growth obstruction for synchronized witnesses). *Let a binary automatic relation have a Q -state representing DFA and let X be the maximum visible input value (zero for an empty tuple). If any witness exists, there is one smaller than $2^Q(X + 1)$. In particular, a single-valued automatic graph cannot define a function with superlinear growth in X .*

Proof. A minimal visible encoding has length n with $2^n \leq 2(X + 1)$. Proposition 5.2 provides a joint word of length at most $n + Q - 1$, so its hidden value is smaller than $2^{n+Q-1} \leq 2^Q(X + 1)$. If the relation is single-valued, that witness is its only output. $\square$

This gives a precise limitation rather than merely a list of unsupported syntax. Neither the graph $y = x^2$ nor the graph $y = 2^x$ over all naturals can have a fixed finite synchronized binary DFA: either growth eventually exceeds every constant multiple of $x + 1$. In contrast, the unary set of powers of two is automatic, and its least member above x grows only linearly. Raising the state budget does not convert the former relations into supported atomic graphs. A different certificate lane is required.

5.3 A regular graph is not automatically a streaming program

Do not advertise the evaluator as a one-pass digit transducer. It first collects reachable states and predecessor information, chooses a successful terminal continuation, and backtracks. Earlier output digits can depend on information arriving later in the input. Regularity of a relation does not, by itself, establish the existence of a deterministic online transducer producing output without lookahead.

For a Lean function, the conservative port is this finite dynamic program, together with its correctness proof and bounded tail traversal. An optional later pass may recognize relations admitting a simpler online or closed-form implementation. Such an optimization must prove equivalence to the checked graph; it is not required for witness existence or for executing the synthesized selector.

5.4 Five executed selector families

A witness longer than its input. For $R(x, y) \equiv y = 2x + 1$, the least-witness graph has three states after minimization. The evaluator returns $2x + 1$, including at the empty encoding of zero. This family is also the diagnostic test for missing zero-tail closure.

A quotient represented relationally. Use $R(x, y) \equiv \exists r (x = 7y + r \wedge r < 7)$. Its least graph has nineteen states. The evaluator returns $\lfloor x/7 \rfloor$, although neither division nor this closed-form implementation is programmed into the graph compiler. The inner remainder quantifier is eliminated before witness extraction.

The next power of two. Use $R(x, y) \equiv x < y \wedge \text{PowTwo}(y)$. The least graph has four states. Its mathematical output is $2^{\text{bitlength}(x)}$, with $\text{bitlength}(0) = 0$. This selector is obtained from a non-Presburger digit predicate and illustrates why a finite collection of affine witness templates is the wrong abstraction for this lane.

The next same-parity integer. Use $R(x, y) \equiv x < y \wedge (\text{EvenPop}(x) \iff \text{EvenPop}(y))$. Its least graph has five states. The result depends on binary digit structure, not just a single residue modulo a fixed small modulus. No explicit formula for the selector is needed by the evaluator.

A partial Diophantine selector. Use $R(x, y) \equiv \exists b (x = 5y + 7b)$. The least graph has twenty-four states. Some small inputs have no witness; the evaluator returns **None** on those inputs rather than inventing a default. An independent arithmetic oracle uses the least possible coefficient $y = (3x) \bmod 7$ and tests $5y \leq x$. The last inequality is essential: a residue condition alone would admit negative values of the other coefficient.

The last source’s largest intermediate DFA has 1,102 states. Reporting only its twenty-four-state output would hide much of the cost. The complete construction certificate contains 8,644 transition cells. The next-power and next-parity examples have small final machines, but that observation is specific to these fixtures, not a general compression theorem.

5.5 Towards alternating witness strategies

The same construction suggests a concrete further integration for a prenex goal

$$\forall x \exists y \forall z \exists u \psi(x, y, z, u).$$

Compile $R_1(x, y) := \forall z \exists u \psi(x, y, z, u)$, then its least graph to obtain $y = f(x)$. Separately compile the least graph for $R_2(x, y, z, u) := \psi(x, y, z, u)$ in output u . Evaluate the latter only at $(x, f(x), z)$; the first selector’s specification proves those calls lie in its domain. The resulting strategy is $u = g(x, f(x), z)$.

For longer finite prefixes, process the existential positions against their remaining suffix formulas, retaining the already selected parameters. The proof is induction on the quantifier prefix. This avoids the unsound shortcut of finding one tuple of witnesses and treating it as a strategy valid against every later universal choice.

Evidence status: The component operations needed for this construction are implemented. A public prenex-strategy compiler, its composition receipts, and Lean strategy extraction are not implemented or separately tested in this bundle.

This is a specific next algorithm, not part of the 643 executed witness traces. Its main engineering challenge is controlling the repeated suffix compilations and making the domain proofs explicit at each selector call.

6 Libraries and a concrete Lean integration plan

6.1 Use established mathematics without inheriting unsupported trust

The first implementation should keep external automata search untrusted and build a small Lean replay layer. It should not port the Python producer wholesale into the kernel, and should not treat a successful Walnut query as a Lean proof.

Mathlib: the semantic substrate. Use `Mathlib.Computability.DFA` and `Mathlib.Computability.NFA` for the language-level semantics. The documented interfaces include `DFA.evalFrom`, Boolean language identities, and `NFA.toDFA_correct`. These provide the vocabulary and established closure results for the port [12, 13]. Their general state types and set-valued transitions still need a proved bridge to efficient finite tables; they are not a ready-made implementation of this certificate format.

The proposed finite representation is a structure with an explicit state count, a state in `Fin Q`, a total transition function on `Fin Q` and binary letters, and a finite accepting bitmap. A decoder must establish array lengths and index bounds before constructing these objects. A table-to-Mathlib interpretation can then be defined once.

Sheng/Aeacu2: projection and encoding proofs. The inspected `Automata/Projection.lean` contains `correct_project`, `project_correct`, and `project_iff`, relating zero-corrected projection to existential quantification [9]. The inspected project pins Lean v4.23.0, whereas Forge’s baseline records v4.34.0 [10, 2]. Its addition file defines an automaton but contains no semantic correctness theorem in that file [11]. Reuse should therefore begin with a source and build audit, not with a claim that the entire arithmetic bridge is already available.

Most importantly, its leading-zero conventions do not match this prototype’s least-significant-first zero tails. Either retain that project’s encoding and adapt the producer, or prove an explicit reversal/encoding transport theorem. Mixing the two libraries by matching similarly named functions would invalidate the quantifier bridge. The source was inspected here but not compiled in this environment.

Walnut: proposer and differential reference. Walnut is a concrete candidate for a future search backend or independently implemented comparison oracle; its scope includes first-order arithmetic with automatic predicates [7, 6]. An adapter would have to fix the numeral system and digit ordering, export the complete atomic/Boolean/projection derivation or independently certify its resulting automata, and bind that derivation to the original query. The present experiment suite does not run Walnut. Its name is not an acceptance receipt.

Lean core arithmetic and finite data. The carry checker needs exact integers, division by positive two, finite lists, finite sets or bitmaps, and Boolean computation. Prove the generic carry and projection theorems once; use ordinary finite evaluation to establish particular table equations. No floating-point rank or numerical optimization library is needed for this worker. A first port can avoid extra native dependencies entirely.

6.2 The minimal theorem stack

A practical port can be organized into the following modules. These are proposed module names, not files already present in Forge.

Proposed module	Required result
Forge/Automatic/Encoding	Digit evaluation, appending zeros, and existence of an encoding for every natural tuple
Forge/Automatic/Table	Interpretation of bounded tables as complete DFAs
Forge/Automatic/Atoms	Label-check soundness for the three arithmetic and four digit atoms
Forge/Automatic/Receipts	Product, complement, quotient and exact-tail local soundness
Forge/Automatic/Quantifier	Subset semantics and existential correctness on all encodings
Forge/Automatic/Check	Well-typed backward graph replay and root verdict soundness
Forge/Automatic/Witness	Prefix dynamic program, bounded tail traversal, and least-choice specification
Forge/Automatic/Reify	A proof connecting the accepted Lean expression to the arithmetic syntax
Forge/Automatic/Tactic	Worker invocation, replay, and insertion of the resulting proof term

The essential theorem interface is schematically

$$\text{check}(q, c) = \text{true} \implies \begin{cases} \forall \rho, \text{Sem}(q, \rho), & c \text{ claims validity,} \\ \neg \text{Sem}(q, \rho_c), & c \text{ supplies a counterexample.} \end{cases}$$

It should be expressed with separate typed result constructors rather than a string tag determining the theorem’s conclusion. A witness theorem has the additional premise that its source automaton has already been proved to represent the intended relation. A theorem about a DFA run alone must not be exported as a theorem about an unrelated Lean predicate.

The manuscript proof structure gives a dependency order. Encoding and atom invariants precede graph replay. Exact tail closure precedes existential compilation. Root verdict soundness uses representation surjectivity. Least-choice correctness uses the order on naturals. This order avoids a large mutually recursive correctness proof spanning reification, search, and execution.

6.3 Reification must preserve the actual goal

A reifier should recognize elaborated Lean expressions, not parse pretty-printed terms. For a supported arithmetic atom it returns both a syntax object and a theorem equating its semantics to the source proposition. For a Boolean operation or binder it constructs the corresponding theorem compositionally. Context tracks are tied to actual bound variables and local constants; a displayed variable name is insufficient evidence of identity.

Constant multiplication may be normalized into coefficients using ring identities over integers. A natural inequality can be transported into the integer interpretation using proved coercion lemmas. Truncated subtraction, modulus conventions, coercions in the opposite direction, and dependent arguments require explicit treatment. Unsupported syntax remains a native Lean residual obligation or causes this lane to decline; it must never be silently approximated and later reported as an equivalence.

Leant and Djex already emphasize retaining source selections and checking actual reconstructed candidates rather than trusting erasures or renderings [14, 15]. Forge has already absorbed

much of that discipline. The specific application here is narrow: preserve the source predicate, the ordered numeric context, and each automatic-atom bridge all the way to the checked root. The Python expected-query comparison is only a prototype analogue of that requirement; it is not a dependent-type-aware Lean reifier.

6.4 Proof evaluation and axiom policy

There are two plausible early replay implementations. One builds explicit proof terms for the local table equations and applies the generic soundness lemmas. The other defines a reflected Boolean checker and proves its soundness, then evaluates it on certificate data using the project's accepted evaluation mechanism. Both need actual compilation and an audit of the generated theorem's dependencies.

A fast native evaluator must not be substituted silently for a requested kernel-only trust profile. The integration should record how evaluation is justified and inspect `#print axioms` on the final theorem, not merely on helper lemmas. The code delivered here makes no claim about that audit: it contains neither a compiled checker nor a compiled `forge` extension.

The initial acceptance milestone should be *small but end to end*: a single Lean theorem with an existential witness longer than its input, reified from its original statement, replayed through the corrected projection certificate, with no `sorry`, new axiom, or unproved external conclusion. An abstract word theorem without the original natural-number bridge does not close that milestone.

6.5 Routing within Forge

This lane should not replace efficient native arithmetic on routine goals. A proposed selection policy is:

```
if the goal is ordinary quantifier-free linear arithmetic:
    prefer the native arithmetic worker
else if all atoms have exact automatic bridges
    and active-track / predicted-table budgets are acceptable:
    invoke the automatic-arithmetic worker
else:
    leave this worker inactive

on a candidate certificate:
    replay it against the original typed query
    insert the resulting theorem only after successful replay
on unsupported syntax, search exhaustion, or replay refusal:
    preserve the original obligation and the diagnostic status
```

The distinguishing trigger is an unbounded number quantifier, a digit-automatic predicate, or a request for a regular witness graph. Pure Boolean size is not sufficient reason to translate a goal here. Conversely, a fixed-width bit-vector reduction is not an equivalent substitute for the unbounded natural-number query unless a separately proved bound makes it so.

A worker result can be a proved relational lemma rather than closure of the entire goal. For example, a source theorem that an automatic predicate has a later occurrence can feed a higher-level construction in Forge. This preserves the existing repository architecture; it does not require a new global solver-combination theorem.

7 Executed experiments and what they establish

7.1 Environment and reproduction

The suite ran under Python 3.13.5 on Linux x86-64 with glibc 2.41. All producer and replay code uses the standard library, and the complete experiment command also runs under `python -S`. The random seed is 20260915. The bundle retains two successful run directories; their summary counts agree exactly. They are repeated executions of the same cases, not two independent corpora and not results to add together.

The first successful run supplies the tables below. Its machine-readable evidence is in `results/run1/`; `results/run2/` retains the post-interface-guard rerun. Search-free replay in a fresh process accepted all 90 stored certificates and all 643 stored witness traces, with both `producer` and `formula` absent from the imported-module set. This confirms executable separation from search, not correctness of the checker implementation.

No Lean, Mathlib, Walnut or other prover was executed for this experiment. The recorded small Python timings exclude operating-system startup and external integration. They are single-run measurements, not a reliable performance ranking or a projected Lean runtime.

7.2 Twenty-five named queries

The named suite has 20 expected true universal closures and 5 expected false ones. All results match those expectations. The fixtures include the alternating cuts in (1), quotient/remainder existence for three divisors, the representability threshold for coins of values 5 and 7, digit-parity identities, unbounded automatic sets, and mixed arithmetic/bitwise relations.

Query	Verdict	Nodes	Cells	Bytes
Double successor total	T	6	24	1,665
No greatest natural	T	7	26	1,917
Greatest natural (false)	F	6	25	1,652
Successor cut	T	12	156	4,529
Half cut	T	12	234	4,922
Division by 3	T	10	274	4,232
Division by 7	T	10	618	6,167
Division by 17	T	10	1,634	12,923
Residue above any bound	T	9	230	4,591
Signed inequality identity	T	5	148	1,884
Congruence translation	T	9	828	7,255
Coin threshold 24	T	9	352	4,983
Coin threshold 23 (false)	F	9	376	5,171
Powers of two unbounded	T	9	66	2,746
Every natural a power (false)	F	1	6	321
Odd power is one	T	8	50	2,319
Thue–Morse even index	T	8	84	2,322
Thue–Morse odd index	T	9	92	2,598
Parity alternates (false)	F	9	112	2,712
Even popcount unbounded	T	8	52	2,600
Same parity later	T	12	88	4,024
Bitwise AND commutes	T	4	56	1,138
XOR recovery	T	5	72	1,394
Carry-free addition parity	T	17	704	6,713
All addition parity (false)	F	11	280	3,854

The four nonempty counterexample tuples are 23 for the incorrect coin threshold, 0 for “every

number is a power of two,” (1, 2) for the incorrect assertion that popcount parity flips at every increment, and (1, 1, 2) for the incorrect unrestricted addition-parity identity. The remaining false query is the closed assertion that a greatest natural exists, so its free-variable tuple is empty.

The coin controls are particularly useful. The query with threshold 24 is accepted, while weakening it to 23 returns the actual obstructing integer 23. The test therefore exercises a quantified Diophantine relation and its implication guard rather than merely recognizing a hard-coded positive theorem.

The root of a closed sentence often minimizes to one state. That does not mean its proof took one state of work. Certificate nodes and transition cells record the intermediate construction. The byte counts in the table are compact JSON payload sizes, excluding the trailing newline on disk.

7.3 Independent numerical and quantified oracles

A separate family generates 180 quantifier-free automata. Each is compared with ordinary integer arithmetic or bit counting on all 16^2 assignments to two visible variables: 46,080 comparisons. Each assignment is also checked after appending one and three all-zero letters, giving 92,160 *additional* padding evaluations. These are finite differential tests of the implementation, not a finite substitute for Theorem 3.4.

Sixty more formulas have explicitly guarded alternation

$$\forall x \leq B \exists y \leq B \forall z \leq B \quad \theta(x, y, z), \quad B \in \{2, 3, 4, 5\}.$$

Their internal atoms combine signed linear inequalities, congruences and digit parity. An independently evaluated complete truth table supplies the expected result. The experiment visits 6,480 oracle leaves and obtains 27 true and 33 false formulas, all matching the automata compiler. Because the bounds are part of the formulas, the oracle completely decides these particular tests; it is not pretending to decide unbounded formulas by truncation.

The arithmetic oracle does not invoke the automata transition code. The certificate replay does not invoke the producer’s exploration or minimization. These are two different kinds of independence, and neither eliminates the need for a formal semantic proof in Lean.

7.4 Witnesses and minimality observations

The five least-witness graphs were each evaluated on inputs 0 through 127 and three much larger numbers:

$$2^{64} - 1, \quad 2^{128} + 3, \quad 10^{100} + 37.$$

There were 643 accepted witness records and 12 expected absence results. All fifteen large-input cases produced the expected witness. For small inputs, 37,124 direct checks of smaller candidate values found no smaller satisfying value. The large cases were compared with separate exact arithmetic or simple digit-parity oracles rather than enumerating huge intervals.

Least-witness relation	Root states	Peak states	Nodes	Cells
$y = 2x + 1$	3	6	10	216
Quotient by 7	19	34	18	2,700
Next power of two	4	9	16	368
Next same popcount parity	5	19	21	476
Least coefficient of 5	24	1,102	13	8,644

The stored graph certificates prove the graphs’ complete formula semantics according to the receipt rules. A concrete accepted run therefore has both satisfaction and leastness as part of that graph’s meaning. The finite smaller-value checks are additional testing evidence. The twelve absence outcomes were compared with the arithmetic oracle; separate per-input absence certificates are not stored.

A graph such as $y = 2x + 1$ is not true for *every* pair (x, y) . Its bundle may consequently carry a counterexample to its universal closure while still supplying a perfectly correct compiled relation. Graph-construction certificates and named theorem verdicts are distinct uses of the same format and must not be pooled into one success percentage.

7.5 An intentionally broken projection

The suite constructs a deliberately incorrect same-length-only projection for $\exists y (y = 2x + 1)$, then compares it with the corrected algorithm. The eight visible inputs are

$$0, 1, 3, 7, 15, 31, 1023, 2^{128} - 1.$$

On each minimal encoding, the broken projection rejects despite the obvious witness. Appending one zero makes its answer change. The corrected projection accepts both representations in all eight cases. The regression thus tests the semantic reason for the correction, not only an incidental implementation detail.

A correct implementation should retain these fixtures permanently. A future optimization that drops tail saturation because “all tests used enough bits” can otherwise reintroduce the error while continuing to pass ordinary fixed-width tests.

7.6 Mutation and resource controls

The checker rejected 1,096 intentionally corrupted objects. The distinct mutation categories are recorded rather than hidden behind a single headline.

Mutation class	Rejections
Changed an acceptance flag	215
Changed a nontrivial transition	159
Out-of-range tail rank	21
Invalid root index	25
Certificate supplied against a different expected query	25
Changed a concrete witness value	643
Integer/Boolean confusion; floating coefficient; incomplete table; negative index; cyclic dependency; unknown-as-proof; omitted tail; spurious tail	8
Total corrupted objects tested	1,096

This is not exhaustive adversarial validation. The ordinary transition mutations are selected modifications, not all possible graph perturbations. A nonsemantic metadata change need not be rejected, and a different but valid certificate should be accepted. The tests are designed to invalidate an enforced semantic or structural obligation.

Four separate budget controls exercised limits and raised **BudgetExceeded**, the exception the producer CLI maps to **unknown**. A separate CLI smoke test confirmed that mapping and the checker’s rejection of the resulting non-proof bundle. Seven additional interface tests cover fresh-variable admission, capture avoidance, free-name computation, the zero-input hidden-tail case, a 201-bit-scale witness calculation, and binding to the original query. These seven tests are separate from the 1,096 mutation objects and are not added to that count.

7.7 The minimization ablation is modest, not a speedup claim

For the first nine named queries, the suite runs both with and without minimization. All verdicts agree. Across this exact nine-case slice, the complete certificate graphs contain 3,440 transition cells without minimization and 3,221 with it, while node counts increase from 67 to 82 because quotient receipts themselves are stored. The reduction in cells is about 6.4%, not an order of magnitude.

The corresponding measured aggregate production times were about 3.54 ms without minimization and 3.67 ms with it. Such tiny single-run times are noisy, but they certainly do not support claiming a speedup. This is useful negative engineering evidence: minimizing after every node can spend more work than it saves on small formulas. The conditional policy proposed in §8 should be evaluated rather than assumed beneficial.

7.8 Failed runs and remaining blind spots

The preserved first failed run ends in an assertion that a configured budget should have triggered. Investigation found a bad test fixture: a single atomic formula legitimately fit a one-node budget. The fixture was changed to a negated atom requiring more than one node. This was a test-contract correction, not a discovered soundness flaw in the arithmetic algorithm. The traceback remains in `results/history/attempt1.log`.

A later source review strengthened the least-graph helper to reject a supposedly fresh variable already free in its relation. The seven interface tests and the full second run validate that change on their stated cases. No broad claim of hostile-input hardening is inferred from it.

The main remaining blind spots are formal correctness of the checker, original Lean-expression reification, tactic integration, large-alphabet scalability, deep-alternation blowups, malformed-input resource exhaustion, and a diverse independently curated theorem corpus. These are concrete next tasks. The current evidence is stronger than unexecuted pseudocode, but substantially weaker than a verified Lean automation system.

8 Complexity, next algorithms, and acceptance milestones

8.1 Where the cost actually goes

With k active tracks, the explicit alphabet has 2^k letters. A product may have Q_1Q_2 states; projecting a Q -state DFA may create up to 2^Q subsets. Repeated complementation and projection can make successive quantifier alternations costly. The unbounded-input semantics does not imply tractable complexity.

The tail correction is relatively cheap: there are only two relevant zero-visible edges per source state, so reverse reachability is linear in the source graph size for one eliminated track. Moving this calculation before determinization is a concrete advantage of the design. It avoids doing tail reachability on a potentially exponential powerset graph and gives ranks with the source-state bound.

Atomic carries are finite, but their simple interval bounds depend on *numeric* coefficient magnitudes rather than their bit lengths. Congruence states can also grow with the modulus. A formula with a huge binary-written coefficient may therefore be difficult despite having very little source text. Resource prediction should account for coefficient size as well as syntax size and alternation depth.

The implemented minimizer repeatedly refines states by the pair consisting of their acceptance

bit and their successor partition identifiers. There are at most Q refinement rounds, each inspecting $2^k Q$ transitions, before stabilization. Dictionary operations and tuple construction add real implementation costs. The checker avoids proving refinement optimality by validating only the resulting quotient map.

Replay checks the entire stored derivation. Atomic and quotient checks are essentially linear in their table sizes, apart from integer arithmetic. Projection checks materialize the exact successor image of each stored subset on each letter, which costs in proportion to the subset members examined, not merely the number of output states. The source and target tables, label arrays, and formula payloads all contribute to storage.

8.2 Block existential elimination

A concrete optimization is to combine adjacent existential quantifiers before determinization. For $\exists y_1 \cdots \exists y_m \phi$, project the m -bit hidden block at once:

$$\Delta(q, d) = \{\delta(q, (d, b)) : b \in \{0, 1\}^m\}.$$

Compute the tail closure using the zero-visible transitions for all 2^m hidden letters, then perform one subset construction. The receipt needs a hidden block value at each positive-rank state, complement closure under every hidden block, and exact subset images over the same block alphabet.

The soundness proof is identical to Theorem 3.1, now choosing a common length for all hidden values. This can avoid repeated intermediate determinization. Its local branching cost is 2^m , so a controller should cap block size rather than combine an arbitrarily long block automatically. The current prototype eliminates one variable at a time; the block rule is a specified next implementation, not a measured result.

8.3 Track support and justified miniscoping

The present compiler keeps every active context track in every atom, even when an atom ignores most of them. A support analysis can annotate each subformula with exactly the free tracks it uses. Compile a machine over that smaller alphabet, then add a proved letter map when combining it with a larger context. The receipt is the equation

$$\delta_B(q, d) = \delta_A(q, \text{restrict}_S(d)),$$

plus identical initial and final states. This is a concrete use of the letter-comapping interface rather than an unproved elimination of apparently irrelevant variables.

Scope-safe rewrites can further reduce the active alphabet:

$$\exists y (A \wedge B) \iff A \wedge \exists y B \quad \text{when } y \notin \text{FV}(A).$$

Analogous rules should be admitted through explicit propositional equivalences and freshness premises. Apply them bottom-up when they reduce estimated peak tracks; avoid gratuitous prenex conversion. This targets the 2^k alphabet factor before applying lower-level state optimizations.

8.4 Minimize by expected downstream savings

The ablation suggests a conditional policy. For a candidate quotient, estimate its downstream use count u , source and quotient cell counts C_s, C_q , and the cost of constructing and storing

the receipt. Prefer minimization when a predicted quantity such as

$$u(C_s - C_q) - C_{\text{receipt}} - C_{\text{refinement}}$$

is positive, using these values only for scheduling, never for correctness. Before a determinization, even a moderate state reduction can be valuable; before a final verdict on a tiny graph, it may not be.

The scheduler should record the policy choice and retain an on/off comparison mode. Do not learn a cost rule from the same nine examples and then report its improvement on those examples as independent evaluation. Larger families with changing moduli, coefficients, track counts and alternation depth are needed.

8.5 Bitmaps, tables, and proof sharing

The prototype represents subsets as sorted tuples and replays them as explicit sets. A production producer can replace them with bitmaps and precompute the two successor images for each retained letter. The certificate format can remain explicit initially. Later, compressed bitmaps can be decoded into finite sets with a theorem that decoding preserves membership and bitwise union.

The witness evaluator sorts reachable states at each position for deterministic output. Its $O(nQ)$ transition count therefore coexists with an additional comparison cost of up to $O(nQ \log Q)$ in this Python implementation. An indexed bitmap scan can give deterministic enumeration without comparison sorting. This is a small but concrete distinction between the abstract evaluator bound and the delivered code.

Subformula memoization already shares nodes within one compilation. Cross-goal cache entries need more than identical arithmetic text: their source bridges, contexts, atom definitions and Lean environment dependencies must still be valid. A safe first cache stores automata as *search hints* and reruns receipt verification against the current typed query. Environment-aware certified reuse is a later optimization, not a reason to weaken root binding.

8.6 An implementation sequence with falsifiable gates

Milestone 1: formal semantics and local atoms. Compile encoding, finite tables, and exact equality/inequality atom theorems in the pinned Lean environment. Include negative coefficients, zero, large constants, and a deliberate wrong floor-division control. The acceptance condition is a theorem about the original natural-number atom, not merely a theorem about an integer recurrence.

Milestone 2: exact projection. Compile the tail-receipt theorem and subset correctness. Replay the empty-input case $\exists y (y = 1)$ and the longer-witness family $\forall x \exists y (y = 2x + 1)$. Reject certificates with omitted and spurious tail states. A same-length encoding theorem does not satisfy this milestone.

Milestone 3: original Lean goals. Implement the supported expression reifier and a minimal tactic adapter. Compile the original statements for the alternating cut, division by 17, and the 5/7 coin threshold. Inspect the final theorem’s axiom inventory and keep actual false statements as completed rejection controls. This is the first point at which a native capability gain may be claimed.

Milestone 4: automatic atoms and executable witnesses. Prove the Lean bridges for powers of two and popcount parity. Implement the bounded dynamic-program witness evaluator and prove its specification. Evaluate the next-power and next-parity selectors at the stated large inputs inside Lean. A Python result pasted as a numeral is not a synthesized Lean function, and a graph theorem without an executable witness does not close this gate.

Milestone 5: comparative evaluation. Only after those gates, compare with the relevant native methods on original statements under matched trust and resource settings. Separate arithmetic-only, automatic-predicate, and witness-construction tasks. Record solved statements, semantic refusals, timeouts, peak memory, certificate bytes and replay costs. Report cold and warm caches separately. Fixed-width bit-vector instances belong in a different benchmark class unless an explicit theorem justifies their finite bound.

No duration estimate is warranted from the Python measurements alone. The difficult work is not writing another BFS; it is formalizing the encoding/reification bridge and making the original statement survive the entire pipeline.

9 Conclusion

The most useful addition here is a new *semantic lane*, not another outer scheduler: first-order automatic arithmetic on unbounded naturals, compiled into finite relations with exact replay receipts. Its quantifier rule handles long hidden values correctly, and its least-graph construction turns those relations into concrete witness programs without a fixed algebraic ansatz.

The proposal is deliberately bounded in what it claims. Classical automata theory supplies the mathematics; existing Lean formalizations supply relevant foundations; the delivered Python implementation demonstrates the proposed operational contract and exposes the padding error with direct controls. The next decisive advance is an end-to-end Lean replay of the original quantified statement. The bundle provides the algorithms, equations, tests and evidence needed to make that advance specific and reviewable.

A Running and extending the bundle

The standard-library-only replay command is:

```
cd prototype
python -S replay_all.py ../results/run1
```

It checks the 90 stored construction certificates against separately stored expected queries, then checks the 643 concrete witness traces against their verified source machines. Replaying the corpus does not regenerate it or import the producer.

To run all experiments into a fresh directory and run the small interface suite:

```
python -S experiments.py --out ../reproduced-results
python -S -m unittest test_interfaces
```

Use a new output directory to preserve historical measurements. Reusing an existing directory overwrites its generated files; the research script does not enforce a no-overwrite policy. Do not run the tests with Python's `-O` flag, which removes assertions used by the experiment harness. The independent checker uses explicit rejection checks rather than relying on assertions.

To regenerate and check one certificate:

```
python -S producer.py \
  ../results/run1/queries/successor_cut.json \
  ../reproduced-successor-cut.json
python -S checker.py \
  ../results/run1/queries/successor_cut.json \
  ../reproduced-successor-cut.json
```

The producer's `--max-states` flag can exercise a budget refusal; `--no-minimize` disables quotient generation. The checker expects the query as a separate argument. A bundle claiming `unknown` is deliberately not accepted as a proof certificate.

A minimal Python use of least-witness synthesis is:

```
from formula import And, le, pow2, least_graph
from producer import Compiler, extract_witness
from checker import verify, check_witness

relation = And(pow2("y"), le({"x": 1, "y": -1}, -1))
graph = least_graph(relation, "y", "z")
query = {"context": ["x", "y"], "formula": graph}
certificate = Compiler().bundle(graph, query["context"])
verify(certificate, query)
machine = certificate["nodes"][certificate["root"]]["dfa"]
inputs = [10**100 + 37]
record = extract_witness(machine, inputs)
if record is None:
    raise RuntimeError("expected a witness for this total relation")
check_witness(machine, inputs, record)
print(record["witness"])
```

The `verify` result for this graph need not say its universal closure is valid; nearly all pairs are outside a function graph. What witness extraction needs is the checked graph semantics, not universality of the graph over all input/output pairs.

B Certificate schema at a glance

The complete machine-readable examples are the stored JSON corpus. The following is an outline, not JSON to submit directly:

```
{
  "schema": "forge-auto-1",
  "query": {"context": [...], "formula": ...},
  "nodes": [
    {"kind": "atom", "ctx": [...], "formula": ...,
     "dfa": {"k": ..., "start": ..., "trans": [...],
             "final": [...]},
     "labels": [...]},
    ...
  ],
  "root": <node index>,
  "verdict": {"kind": "valid", "invariant": [...]}
}
```

The alternative verdict is `counterexample` with `word` and `values`. A quantified node has `child`, `subsets`, `tail_rank`, and `tail_choice`; its formula is an `exists` node and its child's context appends exactly that binder. A quotient node has a `mapping` indexed by every child state. Products have `left`, `right`, and `pairs`. Negation has a `child`.

Every formula atom uses explicit coefficient maps or variable names. Coefficient zero may be retained in a map; it does not alter the mathematics, but the exact supplied query must still match. All numbers on the wire are JSON integers, not floats or decimal strings. This is a small research format; long-term canonical serialization, compatibility versioning, and streaming parsing remain future work.

C Review ledger and ownership of claims

Claim or component	Evidence and qualification
Forge baseline	Pinned merged status, README, closure chapter and Boolean chapter; not a rerun of the repository’s historical experiments.
Automata arithmetic	Established prior work; not claimed as a new mathematical method.
Correct quantified projection	Mathematical proof in this article; Python implementation and receipts tested; related existing Lean source acknowledged.
Canonical witness graph	Mathematical least-choice argument and executed five-family prototype; not an arbitrary higher-order Lean synthesizer.
Independent replay	Fresh <code>python -S</code> process, producer modules not imported; checker not formally verified.
Lean library reuse	Documentation and selected source inspection; no local compilation, dependency migration, or axiom audit.
Leant/Djex inspiration	Source-bound candidate discipline; no claim to close Church indexing, universe-polymorphism or higher-rank synthesis milestones.
Performance	Local single-run Python measurements and a small ablation; no speedup against Lean tactics.
Hardening	Several schema checks and mutation controls; no security or denial-of-service certification.

The full Forge commit used for the comparison is

58ea206bd7ad501b930add568151217bcc7f82a2.

The companion repositories’ branch references observed during review were:

Leant	6bf05ad78c467989e68290f2d08bbcd40802d485
Djex	e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5
Aeacu2	8b5fb2dd7e4f32e17f6c4dd43f2d8980649ebc42

The Leant and Djex READMEs were retrieved through the main-branch interface; these recorded branch references aid follow-up but are not a proof of a frozen multi-repository build. The inspected Aeacu2 files were retrieved at the listed commit.

References

- [1] V. Reshetnikov, *Forge: Implementation status*, reviewed commit 58ea206bd7ad501b930add568151217bcc7f82a2. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/STATUS.md>.
- [2] V. Reshetnikov, *Forge README*, same reviewed commit. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/README.md>.
- [3] *Forge*, article chapter “Boolean search, equality exploration, and external engines,” same reviewed commit. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/11-boolean-and-external.tex>.
- [4] *Forge*, article chapter “Finite certificates for unbounded behaviour,” same reviewed commit. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/06-closure.tex>.
- [5] P. Wolper and B. Boigelot, “An Automata-Theoretic Approach to Presburger Arithmetic Constraints,” in *Static Analysis, SAS 1995*, LNCS 983, pp. 21–32, 1995. DOI: https://doi.org/10.1007/3-540-60360-3_30. Author archive: <https://orbi.uliege.be/handle/2268/74877>.
- [6] H. Mousavi, *Automatic Theorem Proving in Walnut*, arXiv:1603.06017, 2016, revised 2021. <https://arxiv.org/abs/1603.06017>.
- [7] J. Shallit, *Walnut Software*, project overview and documentation index, accessed September 2026. <https://cs.uwaterloo.ca/~shallit/walnut.html>.
- [8] Z. (Aeacus) Sheng, *Formally Verifying Automata for Trusted Decision Procedures*, master’s thesis, Carnegie Mellon University, May 2025. https://www.andrew.cmu.edu/user/avigad/Students/sheng_ms.pdf.
- [9] Z. Sheng, *Automata/Projection.lean*, Aeacu2/Automata, commit 8b5fb2dd7e4f32e17f6c4dd43f2d8980649ebc42; especially `project_iff`. <https://github.com/Aeacu2/Automata/blob/8b5fb2dd7e4f32e17f6c4dd43f2d8980649ebc42/Automata/Projection.lean>.
- [10] Aeacu2/Automata, *lean-toolchain*, same commit. <https://github.com/Aeacu2/Automata/blob/8b5fb2dd7e4f32e17f6c4dd43f2d8980649ebc42/lean-toolchain>.
- [11] Z. Sheng, *Automata/Addition.lean*, same commit. <https://github.com/Aeacu2/Automata/blob/8b5fb2dd7e4f32e17f6c4dd43f2d8980649ebc42/Automata/Addition.lean>.
- [12] The Mathlib community, *Mathlib.Computability.DFA*, documentation accessed September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/DFA.html.
- [13] The Mathlib community, *Mathlib.Computability.NFA*, documentation accessed September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/NFA.html.
- [14] V. Reshetnikov, *Leant README and synthesis capability overview*, main branch observed September 2026. <https://github.com/VladimirReshetnikov/Leant>.
- [15] V. Reshetnikov, *Djex README and source-typed synthesis overview*, main branch observed September 2026. <https://github.com/VladimirReshetnikov/Djex>.